DeepGEMM

One tensor-core kernel library for a whole MoE model

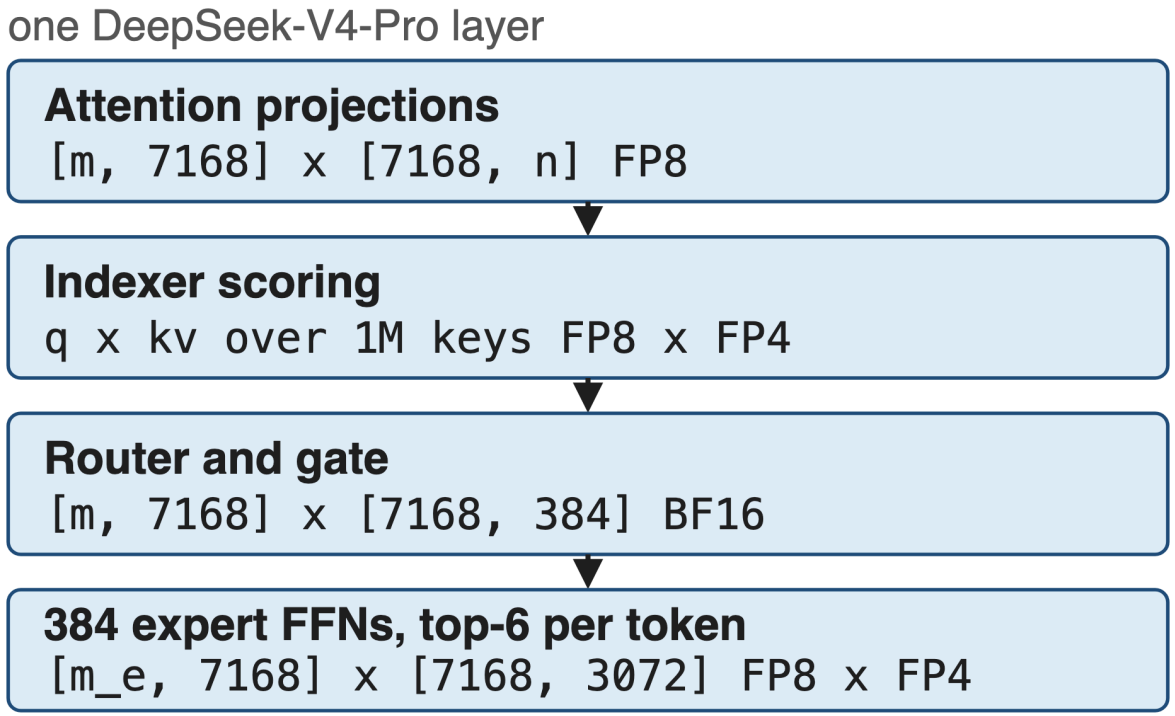
DeepSeek-AI · DeepGEMM v2.8.1

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Frameworks & Platform Stacks · github.com/deepseek-ai/DeepGEMM

One MoE layer, four different matmuls

- DeepSeek-V4-Pro: **1.6T** parameters, 49B active, 1M context
- Each op brings its own dtypes, scales and shapes



Expert-tuned FP8 kernels left performance unclaimed

2.7x

DeepGEMM over an internally tuned CUTLASS 3.6 FP8 GEMM at the V3 decode shape M=64, N=2112, K=7168

DeepGEMM README @ commit 8002b769 (2025-03-25), H800 SXM5, NVCC 12.8

A BLAS call cannot express an MoE step

	A BLAS FP8 GEMM call	One DeepSeek-V4 MoE step
Scaling	one scale per tensor	one scale per 1x128 tile along K
Shapes	M, N, K fixed on the host	tokens per expert known on the GPU
Operands	one dtype for A and B	FP8 activations x FP4 weights
Around it	matmul only	EP dispatch, SwiGLU, combine

Per-group scaling along K is "not directly supported" by a standard FP8 GEMM (DeepSeek-V3 §3.3.2)

Insight: these kernels differ in scheduling, not arithmetic

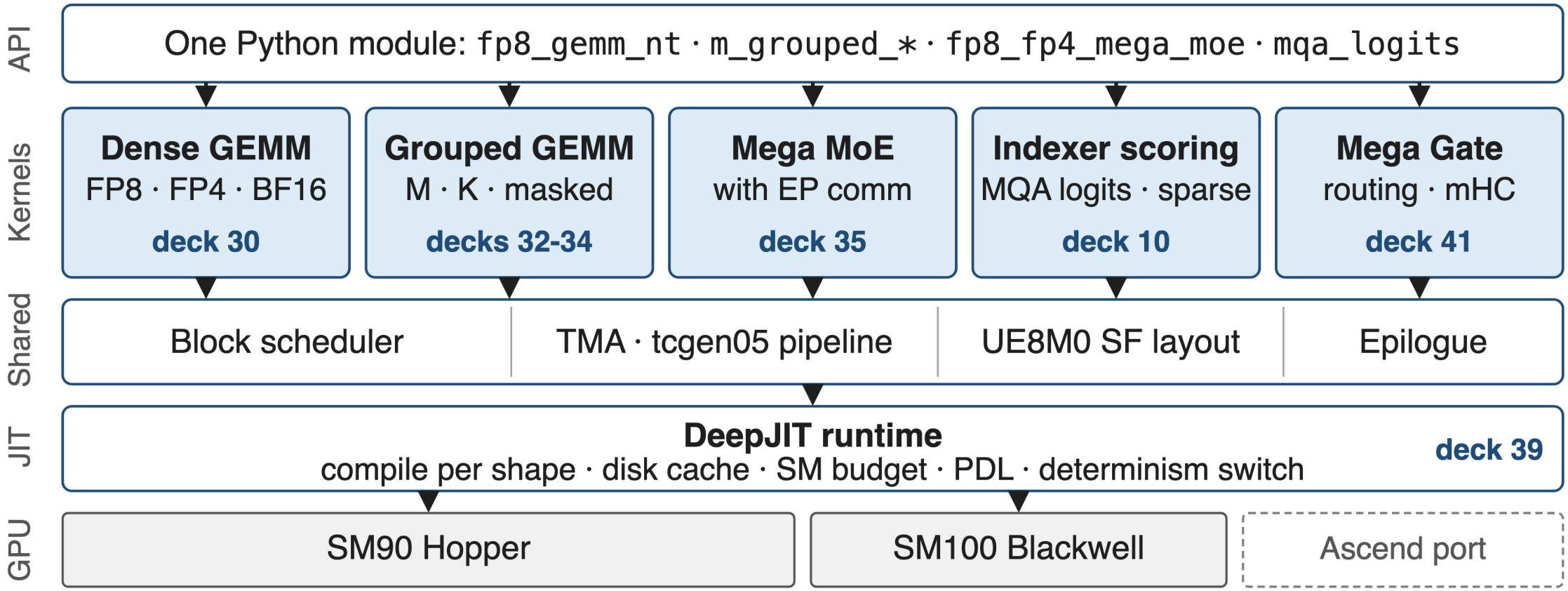
- Same inner loop: TMA load, tensor-core MMA, scaled accumulate, store
- What differs: tile scheduling, scale layout, what surrounds the GEMM

Insight: these kernels differ in scheduling, not arithmetic

- Same inner loop: TMA load, tensor-core MMA, scaled accumulate, store
- What differs: tile scheduling, scale layout, what surrounds the GEMM

Write one kernel per family, share scheduler and layouts, and let a runtime JIT burn every shape and switch into it as a constant

DeepGEMM: five kernel families on one runtime



19 kernel files, 9,593 lines of device code (derived from the repo); each family has its own deck

One GEMM family covers FP8, FP4 and BF16

- `fp8_gemm_{nt,nn,tn,tt}`: SM90 is NT-only, SM100 takes all four majors
- Scales: FP32 on SM90, **4 packed UE8M0** per int32 on SM100, TMA-aligned
- FP8 activations x FP4 weights share the same kernel

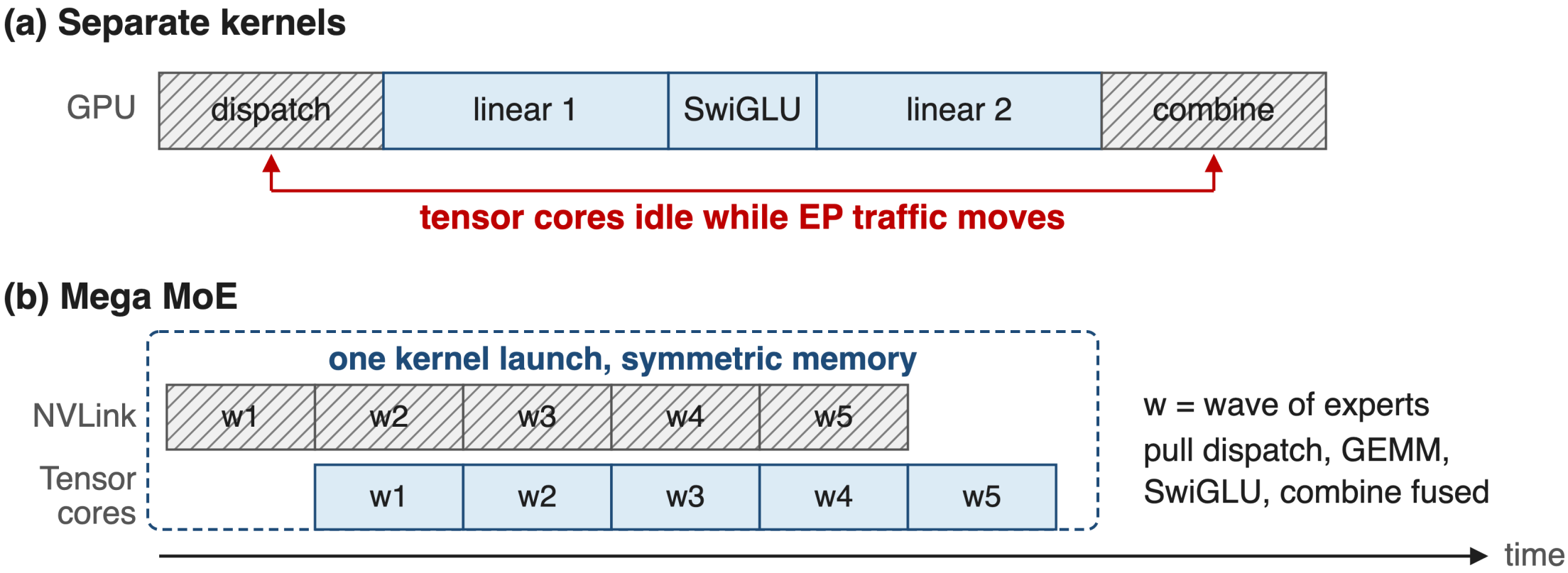
Deep dive: deck 30 (dense GEMM + UE8M0 layout), deck 39 (DeepJIT, PDL, determinism)

Grouped layouts carry per-expert token counts

- **M-grouped contiguous**: experts concatenated, each segment aligned to the M block
- **Masked**: fixed slots plus a mask, so CUDA Graph decode needs no host count
- **K-grouped**: the MoE weight-gradient shape, groups along K

Deep dive: decks 33 (contiguous), 34 (masked), 32 (K-axis)

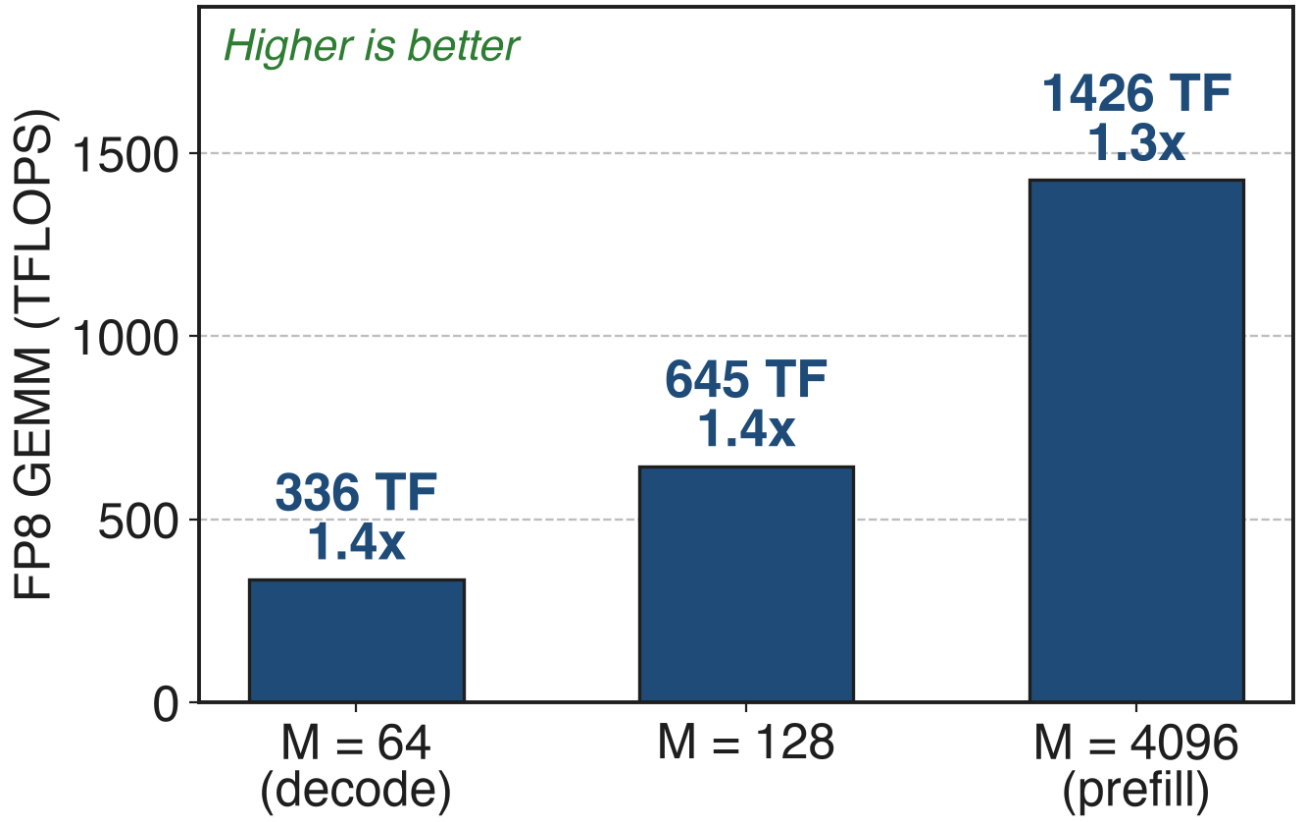
Mega kernels swallow what surrounds the GEMM



Mega Gate does the same for routing, Mega mHC for hyper-connections + RMSNorm

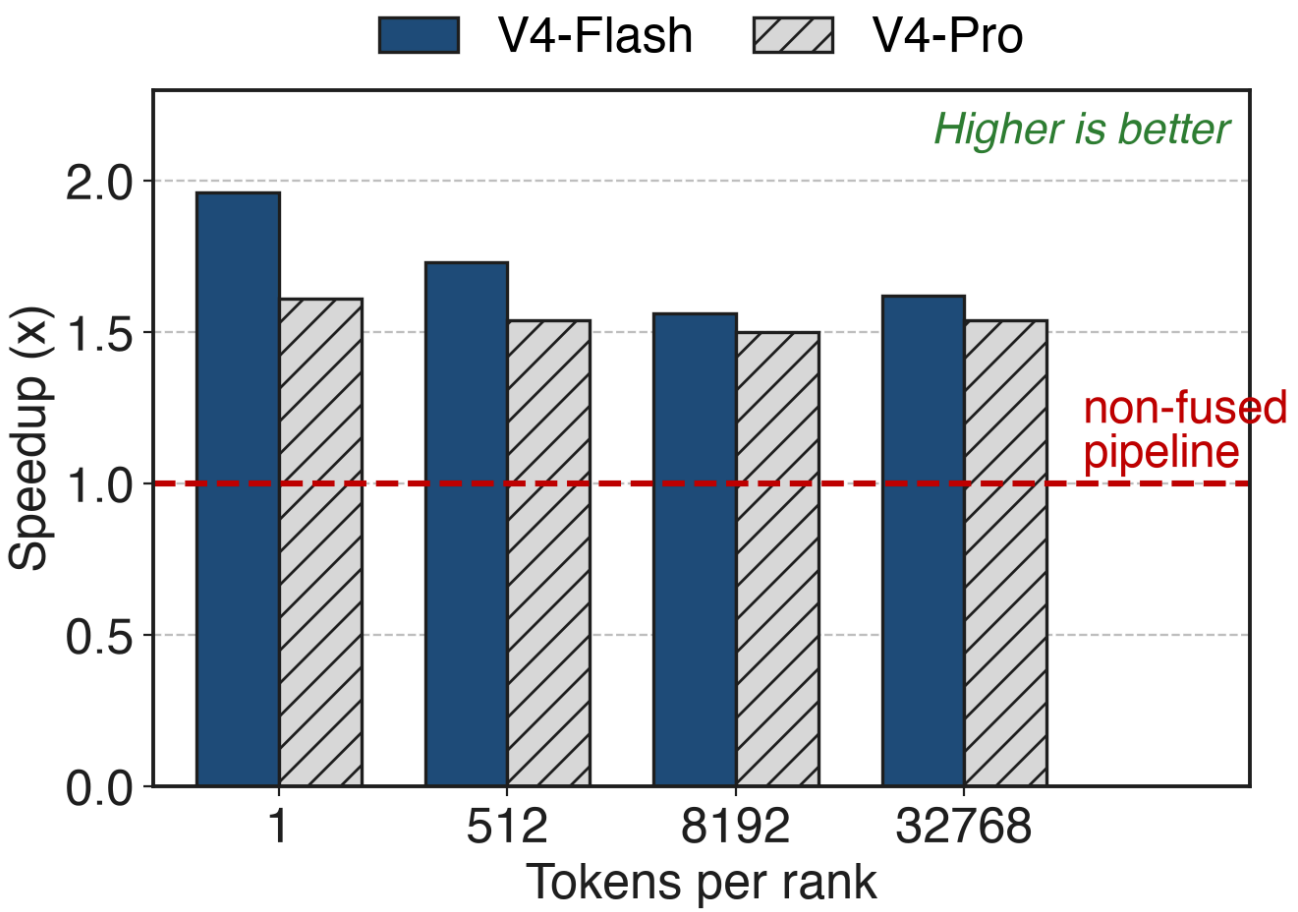
Dense FP8 reaches 1426 TFLOPS on H800

- **1.3-1.4x** over the tuned CUTLASS 3.6 implementation
- N=7168, K=16384; peak over all shapes: **1550 TFLOPS**
- Same kernel from decode M=64 to prefill M=4096



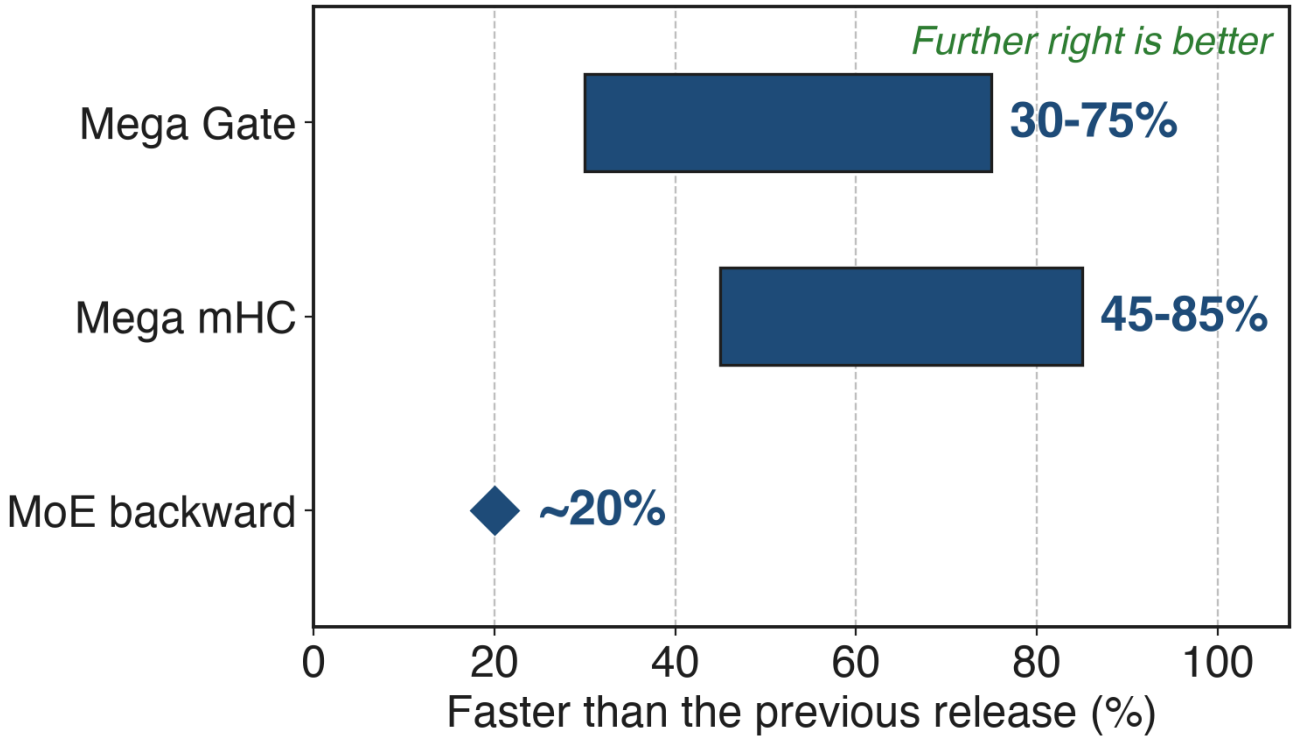
Mega MoE runs 1.50-1.96x the non-fused pipeline

- 8-way expert parallelism, averaged over 8 ranks
- Largest win at **1 token/rank**: RL rollout and agent serving



Fusing the ops around the GEMM pays again

- **Mega Gate**: routing GEMM plus the routing ops
- **Mega mHC**: hyper-connections plus RMSNorm



The library is where DeepSeek-class models land

- SGLang and vLLM ship DeepGEMM paths and warm-up passes
- 8.9k stars; an Ascend port now lives in its own repo

DeepSeek-V4 replaced cuBLAS end-to-end: a BLAS library cannot be batch-invariant (V4 §3.3)

What it costs you

- SM90 or SM100 only, CUDA 12.9+, CUTLASS 4.0+; Ascend is a separate port
- Casting, transposition and scale production stay the caller's job
- Kernels compile on first use; Mega MoE needs symmetric memory, PyTorch 2.9+

Summary

- An MoE step is four different matmuls; a BLAS call fits none of them
- **Insight**: one kernel per family, shapes and switches burned in by the JIT
- Dense FP8 **1426 TFLOPS** (peak **1550**), **1.3-1.4x** over tuned CUTLASS
- Mega MoE **1.50-1.96x**; Mega Gate **30-75%**, Mega mHC **45-85%**

https://github.com/deepseek-ai/DeepGEMM